Perception for ML-Agents (v0.1)

A toolkit for generating computer vision ground truth (Bounding Box, Semantic Segmentation, Depth) and integrating it with ML-Agents as observations for reinforcement learning training.

The intergrated Python sample code is available at: Perception-for-ML-Agents-Sample

Features

Perception provides ground truth CV data for Unity ML-Agents. Allows standard Unity cameras to generate rich Ground Truth data, including Bounding Box, Semantic Segmentation, and Depth Map.

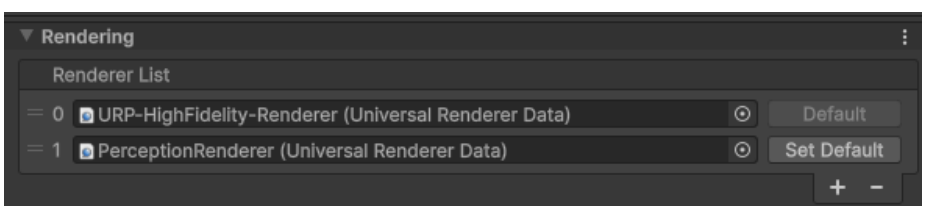
Perception integrates Unity ML-Agents. Encapsulates these ground truth generators as custom sensor components, enabling reinforcement learning agents to directly observe ground truth. Observation data is serialized and sent to the external Python environment via ML-Agents API, thereby achieving complex vision-based reinforcement learning model training.

Perception is a fork of the deprecated com.unity.perception. Stripped of dataset capture (offline data collection) functionality, focusing on lightweight real-time ground truth generation, and supports **Unity 6**.

Require com.unity.ml-agents >= 4.0.0 (ML-Agents Release 23)

Installation

Add `PerceptionRenderer` to the `Rendering -> Renderer List` of the `Universal Render Pipeline Asset` being used.



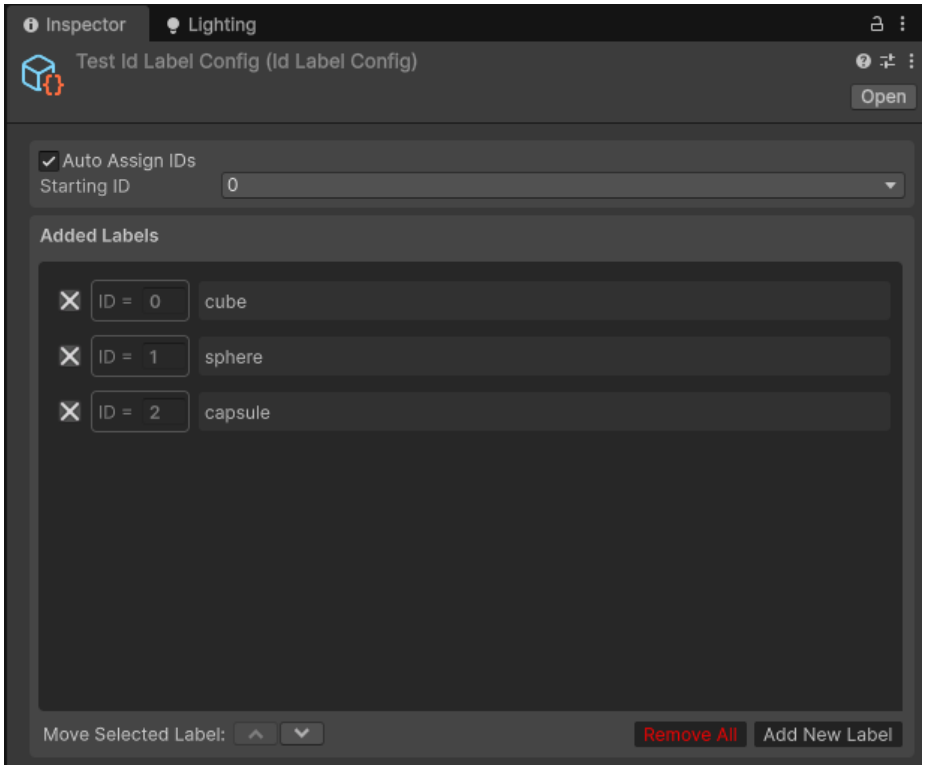
Ensure that the project has the `com.unity.ml-agents>=4.0.0` plugin installed. Installation Guide

Usage

Add Label Configuration

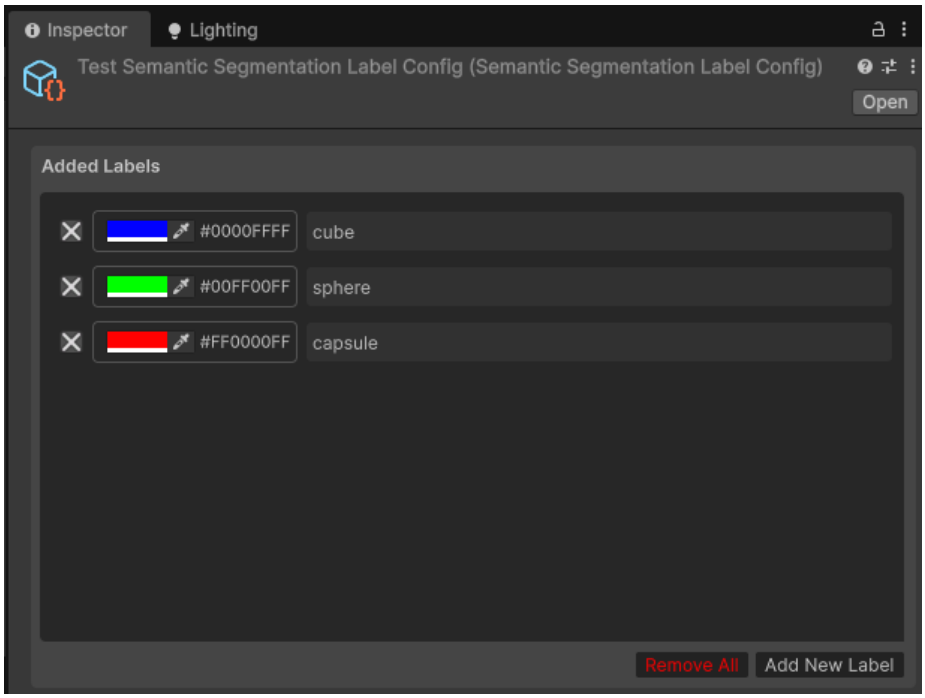
In the `Project` panel, `Create -> Perception -> ID Label Config` to create a new ID label configuration file.

You can specify any label name. It is recommended to turn on the `Auto Assign IDs` option and set `Starting ID` to 0. Since the object category information of BoundingBox is of onehot type, IDs need to be numbered consecutively starting from 0.



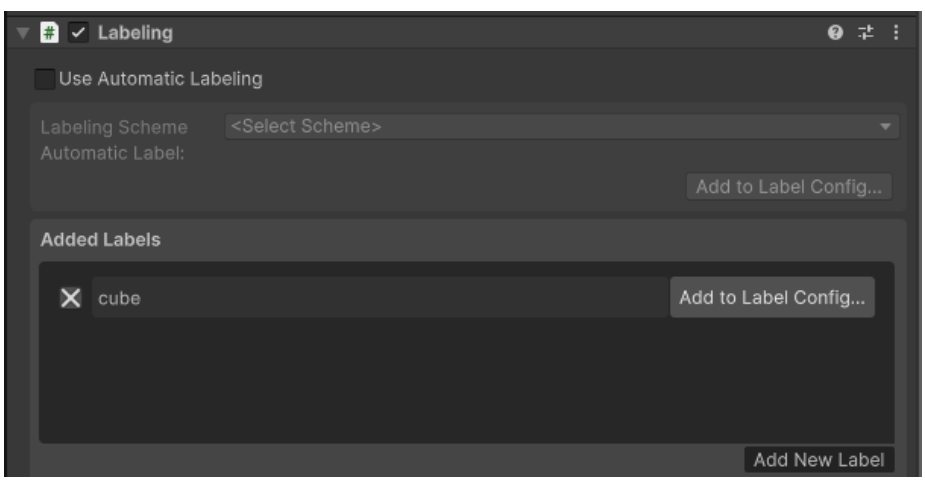
`Create -> Perception -> Semantic Segmentation Label Config` creates a new semantic segmentation label configuration file.

Similarly, you can specify any label name and set any color after semantic segmentation.



Add Label to GameObject

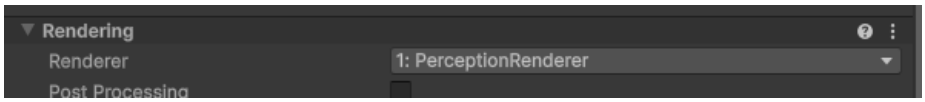
Add the `Labeling` component to GameObjects that need to be labeled for perception. Press `Add New Label` and input the label name in `ID Label Config`.



Using Perception Camera

Set the `Rendering -> Renderer` of the Camera using Perception features to `PerceptionRenderer`.

Pay attention to adjusting the `Near` and `Far` values of `Projection -> Clipping Planes` to ensure the range of the depth map.

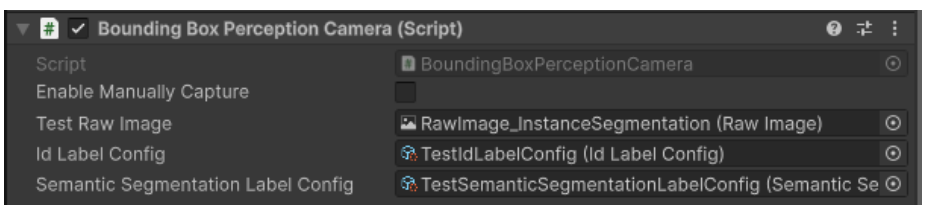


Add specific `Perception Camera` components as needed; multiple can be added to the same Camera.

`Bounding Box Perception Camera`

Used to generate BoundingBox; the underlying implementation uses `Instance Segmentation`, so an ID label configuration file is required.

Configuration Parameters: * `Enable Manually Capture`: Whether to manually call the Capture method for capturing; defaults to automatic capture every frame. Used in ML-Agents components, manual capture is usually not needed, so set to default `false`. * `Test Raw Image`: Whether to display the original Instance Segmentation image in the Canvas's `Raw Image` for debugging or visualization. The displayed Instance Segmentation image assigns a unique color to each object, but it is not observation data for ML-Agents, only for reference. * `Id Label Config`: ID label configuration file. * `Semantic Segmentation Label Config`: Semantic segmentation label configuration file; assigns the semantic segmentation color corresponding to the label in `boundingBoxInfos`.

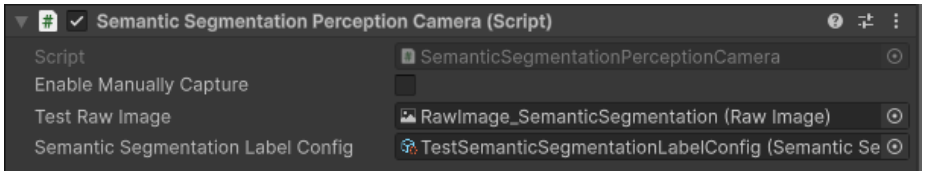


`Semantic Segmentation Perception Camera`

Used to generate semantic segmentation images; the underlying implementation uses `Semantic Segmentation`, so a semantic segmentation label configuration file is required.

Configuration Parameters: * `Enable Manually Capture`: Whether to manually call the Capture method for capturing; defaults to automatic capture every frame. Used in ML-Agents components, manual capture is usually not needed, so set to default `false`. * `Test Raw Image`: Whether to display Semantic Segmentation in the Canvas's `Raw Image`; same as the observation data passed to

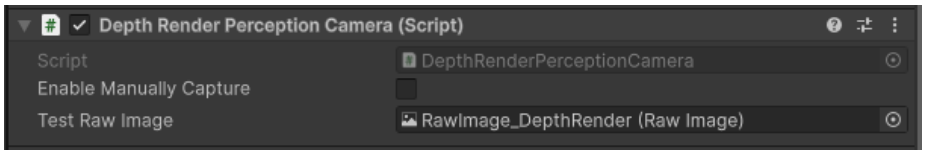
ML-Agents. * `Semantic Segmentation Label Config`: Semantic segmentation label configuration file; assigns the semantic segmentation color corresponding to the label in `boundingBoxInfos`.



`Depth Render Perception Camera`

Used to generate depth images.

Configuration Parameters: * `Enable Manually Capture`: Whether to manually call the Capture method for capturing; defaults to automatic capture every frame. Used in ML-Agents components, manual capture is usually not needed, so set to default `false`. * `Test Raw Image`: Whether to display the depth map in the Canvas's `Raw Image`; same as the observation data passed to ML-Agents.



Integration into ML-Agents

Similar to sensor components provided by ML-Agents, `Perception` provides corresponding sensor components that can be directly added to the Agent.

`Bounding Box Component`

Used to pass Bounding Box information as `BufferSensor` observation to ML-Agents.

Configuration Parameters:

- `Bounding Box Perception Camera`: Bound `Bounding Box Perception Camera` component.
- `Max Number Object`: Maximum supported number of objects; Bounding Boxes exceeding this number will be ignored.
- `Number Object Type`: Number of object categories; should be consistent with the number of categories in the ID label configuration file, determining the length of the onehot category vector in the Bounding Box information.
- `Random`: Ratio of domain randomization; randomly ignores some Bounding Boxes or changes Bounding Box size/position with a certain probability to simulate sensor noise and improve training robustness.
- `Sensor Name`: Sensor name.

- `Image Size`: Sensor image size, `X` is width, `Y` is height.

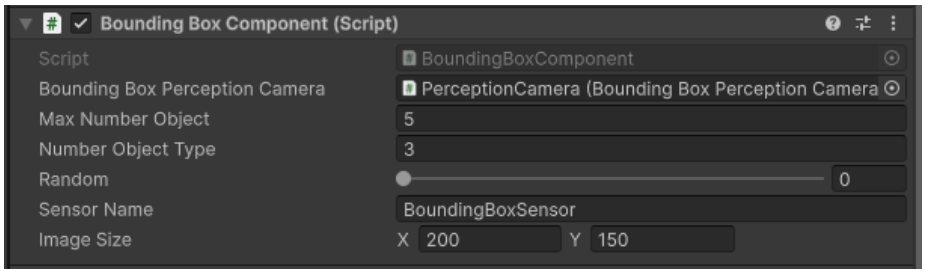
Format of observation received by Python:

A 2D array with dimensions `(Max Number Object, Number Object Type + 4)`, where `Number Object Type + 4` represents the onehot category vector and Bounding Box information for each object.

`[onehot_id_label_vector..., center_x, center_y, width, height]`

Where `center_x, center_y, width, height` are all values normalized to the `[0,1]` range, Coordinate `(0,0)` is the top-left corner of the image, x-axis horizontal, y-axis vertical. Recorded `center_x, center_y, width, height` are the center point and dimensions of the bounding box.

If the number of recognized objects is less than `Max Number Object`, extra rows will be padded with 0.



The observation generated in the example in the figure has dimensions `(5, 7)`, indicating support for up to 5 objects, 3 categories (onehot vector length is 3), plus 4 bounding box information values, totaling 7 dimensions.

`Semantic Segmentation Component`

Used to pass semantic segmentation images as `RenderTextureSensor` observation to ML-Agents.

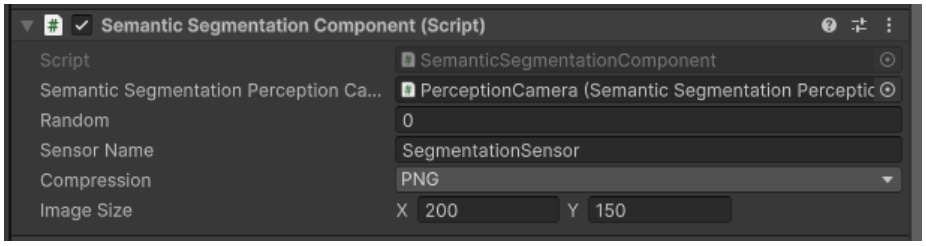
Configuration Parameters:

- `Semantic Segmentation Perception Camera`: Bound `Semantic Segmentation Perception Camera` component.
- `Random`: Ratio of domain randomization; randomly changes object center position with a certain probability.
- `Sensor Name`: Sensor name.
- `Compression`: Sensor image compression format.
- `Image Size`: Sensor image size, `X` is width, `Y` is height.

Format of observation received by Python:

A 3-channel image with dimensions `(3, height, width)`; RGB values of each pixel correspond to color values set in the semantic segmentation label configu-

ration file. Pixels with all `1` in three channels represent background, meaning no object was recognized.



The observation generated in the example in the figure has dimensions `(3, 150, 200)`.

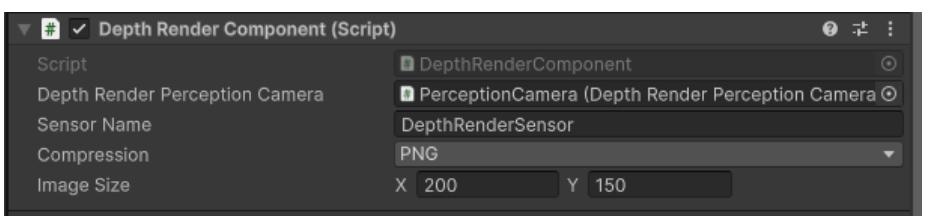
`Depth Render Component`

Used to pass depth images as `RenderTextureSensor` observation to ML-Agents.

Configuration Parameters: * `Depth Render Perception Camera`: Bound `Depth Render Perception Camera` component. * `Sensor Name`: Sensor name. * `Compression`: Sensor image compression format. * `Image Size`: Sensor image size, `X` is width, `Y` is height.

Format of observation received by Python:

A 1-channel image with dimensions `(1, height, width)`; each pixel is a grayscale value in the [0,1] range representing depth information, 0 is near clipping plane, 1 is far clipping plane.



The observation generated in the example in the figure has dimensions `(3, 150, 200)`.

Receiving observation on Python side

On the Python side, ML-Agents will receive and process observation data generated by Perception sensors as Agent observation data. Please refer to the descriptions of each sensor component above for specific observation dimensions and formats.

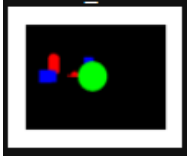
You can refer to the provided `Perception_unity_wrapper.ipynb` Jupyter Notebook file to understand how to receive and process these observation data on the Python side.

The following is a brief example explanation of part of `Perception_unity_wrapper.ipynb`:

```
{'TestPerceptionAgent?team=0': ['BoundingBoxSensor', 'DepthRenderSensor', 'SegmentationSensc
{'TestPerceptionAgent?team=0': [(5, 7), (1, 150, 200), (3, 150, 200)]}
```

Indicates that the Agent `TestPerceptionAgent?team=0` has 3 sensors, namely `BoundingBoxSensor`, `DepthRenderSensor`, and `SegmentationSensor`, with corresponding observation dimensions of `(5, 7)`, `(1, 150, 200)`, and `(3, 150, 200)` respectively.

The following is an example of observation data at frame 3:

```
========== step 3 ==========
[ma_name] : TestPerceptionAgent?team=0
--obs_name-- : BoundingBoxSensor (5, 7)
[[[1.         0.         0.         0.45       0.35333332 0.07000002
   0.06666666]
  [0.         0.         1.         0.19999999 0.38333333 0.08
   0.19333333]
  [1.         0.         0.         0.1575     0.5        0.125
   0.12      ]
  [0.         0.         1.         0.33499998 0.48000002 0.08000001
   0.05333331]
  [0.         1.         0.         0.475      0.5        0.20999998
   0.27999997]]]
--obs_name-- : DepthRenderSensor (1, 150, 200)
```

```
--obs_name-- : SegmentationSensor (3, 150, 200)
```